Informe desafío I

Nicolas Gómez Acevedo

CC. 1034991348

Mariana Marín González

CC. 1028883465

Informática II

Aníbal José Guerra Soler

Universidad de Antioquia

Medellín

2026-1

Introducción

Este proyecto aborda el Desafío I de informática II, cuyo objetivo es desarrollar un motor simplificado del juego tetris en consola, aplicando conceptos avanzados de programación en C++, como operadores a nivel de bits, gestión de memoria dinámica y programación estructurada.

A diferencia del juego tetris tradicional, esta versión no incluye animaciones en tiempo real, sino que opera por turnos: el usuario ingresa una acción (mover, rotar, bajar) y el sistema actualiza el tablero. El juego debe ser capaz de representar el tablero, las piezas, detectar colisiones, eliminar filas completas y detectar condiciones de Game over, todo utilizando exclusivamente operaciones bitwise y memoria dinámica.

Objetivos

- Aplicar operadores a nivel de bits para manipular el estado del tablero y las piezas.

- Gestionar memoria dinámica para crear un tablero de tamaño variable según entradas del usuario.

- Desarrollar una solución modular en C++ sin usar STL, strings ni librerías externas (excepto la random para generar piezas aleatorias)

Metodología

1. Reconocimiento de las dimensiones del tablero y comandos

El módulo de interfaz.cpp implementa las funciones `pedirAncho()` y `pedirAlto()`, encargadas de:

- Solicitar al usuario las dimensiones del tablero.
- Validar que el ancho sea estrictamente un múltiplo de 8 (requisito fundamental para la manipulación de bitwise.
- Validar que ambas dimensiones sean al menos 8 (mínimo establecido).
- Retornar los valores ingresados para ser usados en la inicialización del tablero.

Además de una función leerComando(), donde se implementan validaciones de rango, es decir, las letras validas con las que el usuario puede interactuar con las piezas y con el motor simplificado de tetris.

2. Gestión del tablero

Este módulo centraliza toda la lógica relacionada con el tablero de juego:

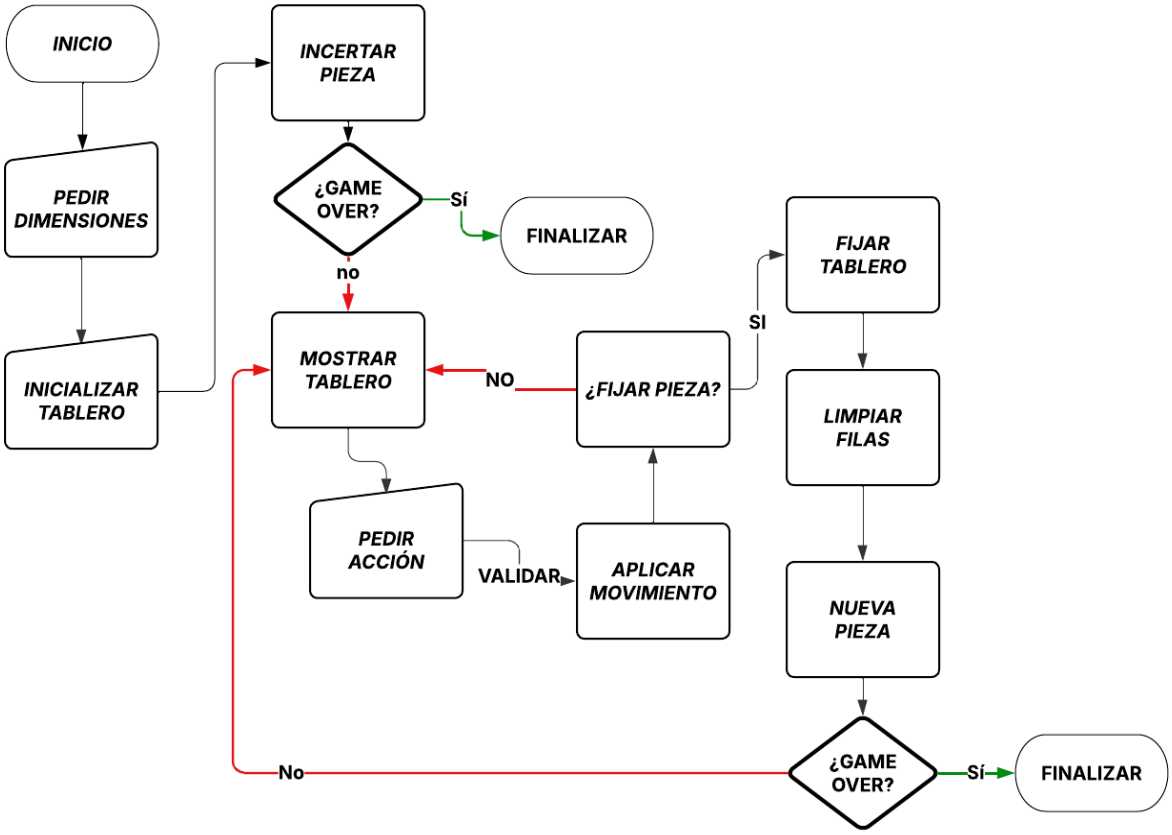
- Inicialización y reserva de memoria dinámica para el tablero según las dimensiones ingresadas.
- Representación visual del tablero en consola, traduciendo los bits en símbolos visuales (´#´ y ´. ´).
- Verificando mediante mascaras si todos los bits de una fila estén en 1, desplazando filas superiores hacia abajo usando aritmética de punteros y limpiando las filas.
- Liberando la memoria reservada para el tablero al final de la partida.

3. Manejo de piezas

El módulo de piezas define y gestiona las figuras del juego:

- Uso efectivo de máscaras que existen dentro del ámbito de las rotaciones de cada pieza para efectos positivos en la eficiencia del programa
- Al usar un tipo de dato que es capaz de almacenar 16 bits se aprovechó para diseñar las piezas en una matriz 4*4 para su impresión visual y para su manejo de rotación
- Para llamar las piezas aleatoriamente utilizamos la función rand(), seleccionando un índice entre 0 y 6 correspondiente a cada pieza y no almacenar mascaras que deban existir durante toda la ejecución del programa, se identifica cada pieza según su índice en el arreglo principal donde están guardadas como un tipo de dato que es capaz de representar 16 bits.

Diagrama de flujo general



Desarrollo del código

La arquitectura del programa se dividió en cuatro módulos.

Módulos	**Función principal**
Main.cpp	Coordina la interacción con el usuario, valida dimensiones, crea el tablero y demuestra todas las funcionalidades del juego.
Tablero.cpp	Implementa la lógica central del tablero incluyendo creación, destrucción, limpieza de filas, colisión, fijar piezas e impresión del estado.
Piezas.cpp	Coordina la interacción con las piezas proporcionando funciones para visualizarlas y rotarlas.
Interfaz.cpp	Controla el flujo completo del juego validando la entrada del usuario, gestionando comandos de movimiento y rotación, verificando límites y condiciones de game over.

Problemas enfrentados y la evolución de su solución

- **Diseño modular:** al inicio fue difícil tomar la decisión de cómo iba hacer la arquitectura del programa sin embargo con asesorías y analizando el problema pudimos tomar de manera adecuada como iba a ser la arquitectura de nuestro programa para la solución del desafío.

- **Manejo eficiente de memoria:** uno de los problemas que tuvimos fue la gestión de memoria para el tablero de juego. Inicialmente consideramos liberar y reasignar memoria cada que se eliminaba una fila completa, pero esto resultaba ineficiente y propenso a errores. La solución al final implementa un arreglo de unsigned long long con manipulación de punteros, que permite reutilizar la misma memoria sin necesidad de operaciones new/delete durante el juego.

- **Manejo de eficiente de máscaras:** cuando nos enfrentamos a las rotaciones de cada pieza tuvimos que tomar decisiones eficientes entre punteros, coordenadas o máscaras, tomamos la decisión de resolver el problema con mascaras que tuvieran un ámbito local en su creación y que su existencia no se prolongara a todo el flujo del programa para no incurrir en un uso ineficiente de la memoria.

Desplazamientos de las piezas: los desplazamientos presentaron sus propias dificultades, ya que había que hacer una reimpresión (quitarla -> validar -> actualizar -> colocarla de nuevo) para evitar que la pieza chocara consigo misma había que quitarla antes de validar el movimiento. También habíamos pensado en el uso de punteros, sin embargo, esto era innecesario ya que solamente iba a ser un recorrido de máximo 16 iteraciones ya que las piezas están siendo tratadas como matrices 4*4, entonces por el tamaño del problema nos basamos en **costo vs beneficio** y nos decidimos por el lado de usar índices inicialmente, además del uso de máscaras binarias para “encender” o “apagar” bits específicos

Justificación de variables y estructuras de control

- unsigned short para las piezas: se utilizó este tipo de dato para representar cada pieza en 16 bits (matriz 4x4), permitiendo manipularlas eficientemente mediante operaciones a nivel de bits.

- unsigned long long* para el tablero: se empleó memoria dinámica para representar cada fila del tablero como un conjunto de bits (hasta 64 columnas), lo que permite una representación compacta y eficiente.

- Memoria dinámica (new[] y delete[]): utilizada para crear el tablero según el tamaño ingresado por el usuario, evitando desperdicio de memoria.

- Variables enteras (int x, y, rotacion, indice): usadas para controlar la posición, orientación y tipo de pieza actual, ya que son valores que representan posiciones, índices y estados que solo pueden tomar valores enteros.

- Ciclos for: empleados para recorrer la matriz 4x4 de cada pieza, facilitando su análisis y manipulación bit a bit.

- Ciclo do-while: utilizado en el flujo principal del juego para mantener la ejecución continua hasta que el usuario decida salir.

- Condicionales if-else: necesarios para interpretar los comandos del usuario, validar movimientos y controlar las rotaciones de las piezas.

- Operadores bit a bit (<<, >>, &, ~): fundamentales para encender, apagar y verificar bits específicos, permitiendo manipular tanto las piezas como el tablero de manera eficiente.

Conclusiones

- La separación en módulos permitió un desarrollo organizado, claro y facilitó el mantenimiento del código.

- La representación del tablero como arreglo de unsigned long long permitió optimizar almacenamiento y operaciones bitwise.

- Expresar las piezas como una matriz 4*4 (16 bits) fue una decisión que ayudó mucho al desarrollo de toda la lógica de las piezas al usar bastantes operaciones bit a bit, implementando una estrategia eficiente y poderosa. Fortaleció la comprensión del funcionamiento de los datos en memoria.

- Crear un modulo de interfaz demostró la importancia de separar la lógica de interacción (entrada de datos) del resto del sistema (lógica del juego), ya que se hace necesario validar las entradas y controlar el flujo del programa por separado.

- El desarrollo del proyecto permitió evidenciar lo importante que es validar cada operación antes de modificar el estado del programa, especialmente porque en el contexto dado se necesita cumplir múltiples condiciones en simultaneo, como la detección de colisiones y movimientos válidos. Esto nos permite mostrar un programa más confiable frente a las entradas del jugador

- Integramos múltiples conceptos fundamentales de la programación, como la modularidad, el manejo de memoria y la manipulación de bits, en una aplicación funcional. Mediante una organización adecuada de código y una toma de decisiones eficientes demostramos que es posible construir sistemas estables con restricciones como las librerías avanzadas